算法基础：习题解答 4

问题 1（30 分）

(1) 设前序遍历为 $p_1, p_2, \ldots, p_n$，中序遍历为 $i_1, i_2, \ldots, i_n$。若当前子树对应的前序区间为 $[preL, preR]$，中序区间为 $[inL, inR]$，则根结点为 $r = p_{preL}$。为了在线性时间内确定左右子树范围，先预处理一个数组（或哈希表）

$$\text{pos}[x] = x \text{ 在中序遍历中的位置.}$$

这样就能在 $O(1)$ 时间内找到根 r 在中序遍历中的位置 $k = \text{pos}[r]$。

于是，左子树的中序区间为 $[inL, k-1]$，右子树的中序区间为 $[k+1, inR]$。左子树结点数为 $L = k - inL$。因此前序区间也可确定为：左子树前序区间为 $[preL+1, preL+L]$；右子树前序区间为 $[preL+L+1, preR]$。

然后递归处理左子树、右子树，最后输出根结点，即可得到后序遍历。若当前子树只有一个结点，则直接输出该结点。

Algorithm 1 Postorder Traversal from the Preorder and Inorder Traversals

Require: The preorder traversal $p_1, \ldots, p_n$ and the inorder traversal $i_1, \ldots, i_n$
Ensure: The postorder traversal sequence O

```
Preprocess pos, where pos[x] gives the inorder position of x
Initialize O as an empty sequence
function DFS(preL, preR, inL, inR)
    if preL > preR then
        return
    end if
    r ← p_preL;  k ← pos[r];  L ← k − inL
    DFS(preL + 1, preL + L, inL, k − 1)
    DFS(preL + L + 1, preR, k + 1, inR)
    Append r to O
end function
DFS(1, n, 1, n)
return O
```

复杂度分析：预处理 pos 需要扫描一次中序序列，耗时为 $O(n)$。递归过程中，每个结点恰好作为某一次递归调用中的根被处理一次，而每次处理除递归调用外只需常数时间，因此所有递归调用中除递归外的总工作量为 $O(n)$。故算法的总时间复杂度为

$$O(n) + O(n) = O(n).$$

(2) 下面给出一个反例，说明在只有前序遍历和后序遍历的情况下，无法确定中序遍历。

第一棵树中，结点 1 的左孩子是结点 2；第二棵树中，结点 1 的右孩子是结点 2：



第一棵树　　第二棵树

那么第一棵树的前序和后序遍历分别为 $1, 2$ 和 $2, 1$，第二棵树的前序和后序遍历也是 $1, 2$ 和 $2, 1$。而它们的中序遍历分别为 $2, 1$ 和 $1, 2$。

所以，如果给定的前序遍历和后序遍历分别为 $1, 2$ 和 $2, 1$，我们无法确定中序遍历是 $1, 2$ 还是 $2, 1$。

问题 2（20 分）

(1) 设当前尚未被覆盖的元素集合为 R。对于每个集合 S_i，记它当前还能新覆盖的元素个数为

$$c_i = |S_i \cap R|.$$

定义该集合当前的单位覆盖代价为 w_i/c_i，其中当 $c_i = 0$ 时，该集合不再参与选择。

每一步选择使 w_i/c_i 最小的集合加入答案，并将它新覆盖的元素标记为已覆盖。重复直到所有元素都被覆盖。

为了优化时间复杂度，维护以下信息：

1. 一个布尔数组 $covered[e]$，表示元素 e 是否已经被覆盖；
2. 对每个集合 S_i，维护当前未覆盖元素个数 $c_i = |S_i \cap R|$；
3. 对每个元素 e，预处理其出现在哪些集合中：$L(e) = \{i \mid e \in S_i\}$；
4. 一个带索引的最小堆，堆中始终维护每个尚未选取且 $c_i > 0$ 的集合当前对应的键值 $(w_i/c_i, i)$。

当某个元素 e 第一次被覆盖时，对于每个 $j \in L(e)$，若 $c_j > 0$，则将 c_j 减一。若 S_j 仍在堆中且减后仍有 $c_j > 0$，则将键值更新为 w_j/c_j；若减后 $c_j = 0$，则删除该集合的堆条目。

也就是说，堆中始终只保留每个集合当前对应的一个条目。

Algorithm 2 Greedy Algorithm for Weighted Set Cover

Require: Sets $S_1, \ldots, S_m \subseteq [n]$, weights $w_1, \ldots, w_m \geq 0$, $\bigcup_i S_i = [n]$
Ensure: A collection of sets covering all elements in $[n]$

$Ans \leftarrow \varnothing$
for each element $e \in [n]$ **do**
    $covered[e] \leftarrow$ **false**; initialize an empty list $L(e)$
end for
for $i = 1$ to m **do**
    $c[i] \leftarrow |S_i|$
    for each $e \in S_i$ **do**
        Add i to $L(e)$
    end for
end for
Initialize an indexed min-heap H
for $i = 1$ to m **do**
    if $c[i] > 0$ **then**
        Insert $(w_i/c[i], i)$ into H
    end if
end for
$tot \leftarrow 0$
while $tot < n$ **do**
    Extract the top entry (key, i) from H
    Add S_i to Ans
    for each $e \in S_i$ **do**
        if $covered[e] =$ **false then**
            $covered[e] \leftarrow$ **true**; $tot \leftarrow tot + 1$
            for each $j \in L(e)$ **do**
                if $c[j] > 0$ **then**
                    $c[j] \leftarrow c[j] - 1$
                    if S_j has an entry in H **then**
                        if $c[j] > 0$ **then**
                            Update the key of S_j in H to $w_j/c[j]$
                        else
                            Delete the entry of S_j from H
                        end if
                    end if
                end if
            end for
        end if
    end for
end while
return Ans

时间复杂度分析：设

$$M = \sum_{i=1}^{m} |S_i|,$$

即所有元素–子集对 (e, S_i) 的总数。预处理阶段需要扫描所有集合及其元素一次，以建立各个 $L(e)$ 并初始化所有 c_i，因此时间复杂度为 $O(n + m + M)$。

在算法执行过程中，每个元素只会在第一次被覆盖时被真正处理一次。处理元素 e 时，需要遍历所有包含它的集合，即遍历 $L(e)$。因此所有这类更新操作的总次数为

$$\sum_{e=1}^{n} |L(e)| = M.$$

若使用二叉堆维护当前最小单位覆盖代价，则每次键值修改或删除的时间复杂度为 $O(\log m)$，这样的修改总共发生 $O(M)$ 次。此外，每次贪心选择需要一次取堆顶操作，共进行至多 m 次，因此这部分复杂度为 $O(m \log m)$。故总时间复杂度为

$$O\big(n + m + M + (M + m) \log m\big).$$

当 $m \geq 2$ 且全集可被覆盖时，$n \leq M$，可简写为 $O(M \log m + m \log m)$。

备注：这里 c_i 随覆盖过程减小，单位覆盖代价 w_i/c_i 随之增大。最小堆需要增加键值；斐波那契堆增加键值一般仍需 $O(\log m)$ 摊还时间，因此使用它时上述总时间上界仍成立。

(2) 设最优解的总代价为 OPT。下面证明上述贪心算法的近似比为

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq 1 + \ln n,$$

从而得出它是一个 $O(\log n)$ 近似算法。

贪心算法每一步都选择当前使 $w_i/|S_i \cap R|$ 最小的集合，其中 R 表示当前尚未被覆盖的元素集合。

我们对每一步选出的集合费用作均摊：若某一步选中集合 S_i，并且它在当前新覆盖了 d 个元素，则把它的费用 w_i 均摊到这 d 个新覆盖元素上，即每个新覆盖元素收取费用 w_i/d。记每个元素 e 最终被收取的费用为 $c(e)$，则贪心算法的总代价恰好等于所有元素所收费用之和：

$$ALG = \sum_{e \in [n]} c(e).$$

因此，只需证明 $\sum_{e \in [n]} c(e) \leq H_n \cdot OPT$。

先证明下面的引理。

引理 1. *设某一步开始时，尚未被覆盖的元素集合为 R，并记 $r = |R|$。则该步贪心所选集合对应的单位覆盖代价至多为 OPT/r。*

证明. 考虑最优解中的所有集合。由于最优解能够覆盖整个全集，因此当然也能够覆盖当前尚未被覆盖的元素集合 R。

设最优解由若干集合组成，它们的总代价为 OPT。这些集合覆盖了 R 中全部 r 个元素。因此，在这些集合中，至少存在一个集合 S_j 满足

$$\frac{w_j}{|S_j \cap R|} \leq \frac{OPT}{r}.$$

否则，设最优解中与 R 相交的集合构成的集合族为 O，对每个集合 $S_j \in O$，如果都满足 $w_j/|S_j \cap R| > OPT/r$，那么对这些集合的总代价进行求和，有

$$\begin{aligned} \sum_{S_j \in O} w_j &> \sum_{S_j \in O} \frac{OPT}{r} |S_j \cap R| \\ &= \frac{OPT}{r} \sum_{S_j \in O} |S_j \cap R| \geq \frac{OPT}{r} r = OPT. \end{aligned}$$

这意味着仅最优解中覆盖 R 的部分就已经超过了 OPT，与最优解的定义矛盾。因此，最优解中至少存在一个集合 S_j 满足 $w_j/|S_j \cap R| \leq OPT/r$。

而贪心算法每一步选择的是使 $w_i/|S_i \cap R|$ 最小的集合，因此该步贪心所选集合一定满足

$$\frac{w_i}{|S_i \cap R|} \leq \frac{OPT}{r}.$$

□

下面证明原问题的结论，也就是贪心算法的总代价至多为 $H_n \cdot OPT$。

证明. 设某一步开始时还有 r 个元素未被覆盖。由上面的引理可知，这一步每个新覆盖元素所分摊到的费用都至多为 OPT/r。

现在按照元素被覆盖的先后顺序来编号，同一步内任意排序。设第 1 个被覆盖的元素收费为 c_1，第 2 个被覆盖的元素收费为 c_2，依此类推，第 n 个被覆盖的元素收费为 c_n。

当还剩 r 个元素未覆盖时，新覆盖的元素收费至多为 OPT/r。因此可得

$$c_1 \le \frac{OPT}{n}, \qquad c_2 \le \frac{OPT}{n-1}, \quad \dots, \quad c_n \le OPT.$$

于是

$$ALG = \sum_{k=1}^{n} c_k \le OPT\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right) = H_n \cdot OPT.$$

□

又因为调和级数满足 $H_n \le 1 + \ln n$，所以 $ALG \le (1 + \ln n)OPT$。因此上述贪心算法是一个 $H_n \le 1 + \ln n$ 近似算法，从而也是一个 $O(\log n)$ 近似算法。

问题 3（30 分）

(1) 为了找到 $\mathbb{R}^d$ 中到 $x_1, x_2, \dots, x_n$ 的欧式距离平方和最小的点，我们考虑如下目标函数：

$$f(y) = \sum_{i=1}^{n} \|y - x_i\|^2 = \sum_{i=1}^{n}\sum_{j=1}^{d} (y_j - x_{ij})^2,$$

其中 $y = (y_1, y_2, \dots, y_d)$ 是我们要寻找的点，$x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ 是给定的点。

对目标函数 $f(y)$ 分别对 $y_1, y_2, \dots, y_d$ 求偏导数，并设置为零，以找到最小值点：

$$\begin{aligned}
\frac{\partial f}{\partial y_1} &= 2\sum_{i=1}^{n}(y_1 - x_{i1}) = 0 \quad \Longrightarrow \quad y_1 = \frac{1}{n}\sum_{i=1}^{n} x_{i1}, \\
\frac{\partial f}{\partial y_2} &= 2\sum_{i=1}^{n}(y_2 - x_{i2}) = 0 \quad \Longrightarrow \quad y_2 = \frac{1}{n}\sum_{i=1}^{n} x_{i2}, \\
\frac{\partial f}{\partial y_3} &= 2\sum_{i=1}^{n}(y_3 - x_{i3}) = 0 \quad \Longrightarrow \quad y_3 = \frac{1}{n}\sum_{i=1}^{n} x_{i3}, \\
&\vdots \\
\frac{\partial f}{\partial y_d} &= 2\sum_{i=1}^{n}(y_d - x_{id}) = 0 \quad \Longrightarrow \quad y_d = \frac{1}{n}\sum_{i=1}^{n} x_{id}.
\end{aligned}$$

由于 $\nabla^2 f = 2nI_d$ 正定，这个点是唯一的全局最小值点。因此点 y 的每一维坐标都等于对应维度坐标的平均值，所以可以直接在 $O(nd)$ 时间内计算出这个点 y 的坐标。

Algorithm 3 Point Minimizing the Sum of Squared Euclidean Distances

Require: Points $x_1, \dots, x_n \in \mathbb{R}^d$，$n \ge 1$
Ensure: A point $y \in \mathbb{R}^d$ minimizing $\sum_{i=1}^{n} \|y - x_i\|^2$

```
for j = 1 to d do
    s ← 0
    for i = 1 to n do
        s ← s + x_ij
    end for
    y_j ← s/n
end for
return y
```

(2) 为了找到 $\mathbb{R}^d$ 中到 $x_1, x_2, \dots, x_n$ 的曼哈顿距离和最小的点，我们考虑如下目标函数：

$$f(z) = \sum_{i=1}^{n} \|z - x_i\|_1 = \sum_{i=1}^{n}\sum_{j=1}^{d} |z_j - x_{ij}|,$$

其中 $z = (z_1, \ldots, z_d)$。这可以转化为对每一维坐标分别求解最小化问题。对于第 j 维，我们需要找到 z_j 使得

$$f_j(z_j) = \sum_{i=1}^{n} |z_j - x_{ij}|$$

最小化。这个其实等于寻找 x_{ij} 的中位数。给出一个简单的证明。

证明. 将 $x_{1j}, x_{2j}, \ldots, x_{nj}$ 按从小到大排序为 $a_1 \le a_2 \le \cdots \le a_n$。

当 $z_j < a_{\lceil n/2 \rceil}$ 时，右侧元素个数不少于左侧，故 $f_j(z_j) = \sum_{i=1}^{n} |z_j - a_i|$ 随着 z_j 增大而不增；当 $z_j > a_{\lfloor n/2 \rfloor + 1}$ 时，左侧元素个数不少于右侧，故 $f_j(z_j)$ 随着 z_j 增大而不减，因此最小值在中位数处取得。

更严格地说：当 n 为奇数时，最优解唯一，为中位数；当 n 为偶数时，任意

$$z_j \in [a_{n/2}, a_{n/2+1}]$$

都能取得最小值，因此算法返回其中任意一个中位数即可。 □

对于第 j 维，可以用找中位数算法在 $O(n)$ 时间内找到 x_{ij} 的一个中位数。对于所有 d 维，总时间复杂度为 $O(nd)$。因此答案是点 z 的每一维坐标都是对应维度的一个中位数。

Algorithm 4 Point Minimizing the Sum of ℓ_1 Distances

Require: Points $x_1, \ldots, x_n \in \mathbb{R}^d$, $n \ge 1$
Ensure: A point $z \in \mathbb{R}^d$ minimizing $\sum_{i=1}^{n} \|z - x_i\|_1$
1: **for** $j = 1$ to d **do**
2: Form the list $A_j = (x_{1j}, x_{2j}, \ldots, x_{nj})$
3: $z_j \leftarrow \textsc{Median}(A_j)$
// $\textsc{Median}(A_j)$ returns a median of A_j.
4: **end for**
5: **return** z

问题 4（20 分）

使用动态规划完成这个问题。对每个结点 u，定义

$$dp[u][1] = \text{在以 } u \text{ 为根的子树中，选 } u \text{ 时的最大权重和,}$$
$$dp[u][0] = \text{在以 } u \text{ 为根的子树中，不选 } u \text{ 时的最大权重和.}$$

则状态转移为

$$dp[u][1] = u.w + \sum_{v \in u.child} dp[v][0],$$

因为若选 u，则它的所有孩子都不能选；以及

$$dp[u][0] = \sum_{v \in u.child} \max\{dp[v][0], dp[v][1]\},$$

因为若不选 u，则每个孩子可选可不选，只需取较优者。

为了输出一个具体解，在计算 dp 的同时记录决策。设 $choice[u][0][v]$ 表示当 u 不选时，对孩子 v 的最优选择：若其值为 1，表示选 v；若为 0，表示不选 v。而当 u 被选时，它的所有孩子都只能不选，因此无需额外记录。这个可以使用链表或哈希表来存储每个结点的决策信息，直接使用二维数组 $choice[u][0][v]$ 也可以，但可能会比较浪费空间。

实现方式是：对整棵树做一次 DFS（后序遍历），先算所有孩子，再算当前结点，并在计算 $dp[u][0]$ 时顺便记录每个孩子的决策。设根为 r，若 $dp[r][1] > dp[r][0]$，则根被选；否则根不选。随后根据记录的决策递归输出所有被选中的结点即可。

Algorithm 5 Maximum-Weight Independent Set on a Rooted Tree

Require: A rooted tree T with root r, node weights $u.w$, and children $u.child$
Ensure: An independent set S whose total weight is maximum

$S \leftarrow \varnothing$
function DFS(u)
    $dp[u][1] \leftarrow u.w$; $dp[u][0] \leftarrow 0$
    for each child $v \in u.child$ **do**
        DFS(v)
        $dp[u][1] \leftarrow dp[u][1] + dp[v][0]$
        if $dp[v][1] > dp[v][0]$ **then**
            $dp[u][0] \leftarrow dp[u][0] + dp[v][1]$
            $choice[u][0][v] \leftarrow 1$
        else
            $dp[u][0] \leftarrow dp[u][0] + dp[v][0]$
            $choice[u][0][v] \leftarrow 0$
        end if
    end for
end function
function PRINT($u, take$)
    if $take = 1$ **then**
        Add u to S
        for each child $v \in u.child$ **do**
            PRINT($v, 0$)
        end for
    else
        for each child $v \in u.child$ **do**
            PRINT($v, choice[u][0][v]$)
        end for
    end if
end function
DFS(r)
if $dp[r][1] > dp[r][0]$ **then**
    PRINT($r, 1$)
else
    PRINT($r, 0$)
end if
return S

时间复杂度分析：在第一次 DFS 中，每个结点都会被遍历一次，并且对于树中的每一条父子边，都恰好处理一次，用于更新对应的动态规划状态以及记录决策信息，因此这一部分的时间复杂度为 $O(n)$。

在随后输出答案的过程中，再对整棵树递归一次。此时每个结点同样至多被访问一次，每条父子边也至多被检查一次，因此输出过程的时间复杂度也是 $O(n)$。

另外，将被选中的结点加入答案集合并输出，总代价至多为 $O(n)$。

综上，设 n 为树中结点的数量，总时间复杂度为

$$O(n) + O(n) + O(n) = O(n),$$